

교과목명: Python프로그래밍응용

학과: AI소프트웨어학과

학번: 2026591016

이름: 원종우

제출일: 2026.07.14

소스코드 작성/편집: Visual Studio Code

1. 문제 설명

AI 소프트웨어학과 학생들의 정보를 관리하고 성적을 처리하는 성적확인 프로그램을 C언어로 구현하는 문제이다. 콘솔 인터페이스를 통해 학생 정보 입력, 개별 성적 입력, 반별 성적 확인, 과목별 최고 득점자 확인, 그리고 정보 저장이라는 다섯가지 주요기능을 구현해야한다. 특히 학생 한 명에 대한 기본 정보와 세 과목의 세 과목의 성적 데이터를 효율적으로 다루기 위해 구조체 중첩 방식을 사용해야하며, 최대20명의 학생 정보를 유연하게 처리하기 위해 구조체를 활용한 동적 메모리 할당(malloc)을 반드시 적용해야 한다.

2. 풀이 방법

조건을 모두 만족하기 위해 먼저 데이터 구조를 계층적으로 설계했다. 파이썬, 자바, C언어 점수와 성적 입력 여부를 확인하는 플래그 변수를 포함하는 하위 구조체인 성적 정보 구조체 grad를 정의하고, 이를 학번, 이름, 나이를 포함하는 상위 구조체인 학생 정보 구조체 student 내부에 포함시키는 구조체 중첩 방식을 적용하면된다. 또한, 프로그램 실행 시 최대 20명의 학생 데이터를 담을 수 있는 메모리 공간을 확보하기 위해 malloc 함수를 사용하여 동적으로 메모리를 할당하였으며, 프로그램 종료 시에는 free 함수로 할당된 메모리를 해제하도록 메모리를 관리한다.

3. 소스 코드

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

#define MAX_STUDENTS 20

// 조건 3. 성적정보(grad) 구조체 정의 (파이썬, 자바, C언어 점수)
typedef struct {
    int python;
    int java;
```

```

int c_lang;
int has_grades; // 성적 입력 여부를 확인하기 위한 플래그
} grad;

// 조건 2, 4 학생정보(student) 구조체 정의 (학번, 이름, 나이, 성적정보 포함 -> 구조체
중첩)
typedef struct {
    char student_id[20];
    char name[20];
    int age;
    grad grades; // 하위 구조체 포함
} student;

// 함수 원형 선언
// 조건 1. 전체 5개의 메뉴 제공 화면 (학생 정보 입력, 개별 성적 입력, 반별 성적 확인,
과목별 최고 득점자 확인, 정보 저장, 프로그램 종료)
void print_menu() {
    printf("\n=====\\n");
    printf(" 성적확인 프로그램 \\n");
    printf("=====\\n");
    printf(" 1: 학생 정보 입력\\n");
    printf(" 2: 개별 성적 입력\\n");
    printf(" 3. 반별 성적 확인\\n");
    printf(" 4. 과목별 최고 득점자 확인\\n");
    printf(" 5. 정보 저장\\n");
    printf(" 0. 프로그램 종료\\n");
    printf("=> 실행할 메뉴를 선택하세요 : ");
}

// 조건 6. 1번 메뉴: 학생 기본정보(학번, 이름, 나이) 입력
void input_student_info(student* list, int* count) {
    if (*count >= MAX_STUDENTS) {
        printf("\\n더 이상 학생 정보를 입력할 수 없습니다. (최대 20명)\\n");
        return;
    }
    printf("\\n[1. 학생 정보 입력]\\n");
    printf("학번을 입력하세요: ");
    scanf("%s", list[*count].student_id);
    printf("이름을 입력하세요: ");
    scanf("%s", list[*count].name);
    printf("나이를 입력하세요: ");
    scanf("%d", &list[*count].age);

    list[*count].grades.has_grades = 0; // 성적 미입력 상태로 초기화

```

```

    (*count)++;
    printf("-> 학생 정보가 정상적으로 등록되었습니다.\n");
}

// 조건 7. 2번 메뉴: 개별 성적 입력 (학생정보가 입력되어 있는 경우에만 가능)
void input_grades(student* list, int count) {
    char target_id[20];
    int found_index = -1;

    if (count == 0) {
        printf("\n등록된 학생 정보가 없습니다. 먼저 학생 정보를 입력하세요.\n");
        return;
    }

    printf("\n[2. 개별 성적 입력]\n");
    printf("성적을 입력할 학생의 학번을 입력하세요: ");
    scanf("%s", target_id);

    // 학번 검색
    for (int i = 0; i < count; i++) {
        if (strcmp(list[i].student_id, target_id) == 0) {
            found_index = i;
            break;
        }
    }

    if (found_index == -1) {
        printf("-> 해당 학번의 학생을 찾을 수 없습니다.\n");
        return;
    }

    printf("-> %s 학생의 성적을 입력합니다.\n", list[found_index].name);
    printf("파이썬 점수: ");
    scanf("%d", &list[found_index].grades.python);
    printf("자바 점수: ");
    scanf("%d", &list[found_index].grades.java);
    printf("C언어 점수: ");
    scanf("%d", &list[found_index].grades.c_lang);

    list[found_index].grades.has_grades = 1; // 성적 입력 완료 상태로 변경
    printf("-> 성적 입력이 완료되었습니다.\n");
}

// 조건 8. 3번 메뉴: 반별 성적 확인 (기본정보 + 교과목별 성적 + 평균)

```

```

void print_all_students(student* list, int count) {
    if (count == 0) {
        printf("\n등록된 학생 정보가 없습니다.\n");
        return;
    }

    printf("\n[3. 반별 성적 확인]\n");
    printf("%-15s %-10s %-5s %-10s %-10s %-10s %-10s\n", "학번", "이름", "나이", "파이썬", "자바", "C언어", "평균");

    printf("-----\n");

    for (int i = 0; i < count; i++) {
        printf("%-15s %-10s %-5d ", list[i].student_id, list[i].name, list[i].age);

        if (list[i].grades.has_grades == 1) {
            double avg = (list[i].grades.python + list[i].grades.java + list[i].grades.c_lang) / 3.0;
            printf("%-10d %-10d %-10d %-10.2f\n", list[i].grades.python, list[i].grades.java, list[i].grades.c_lang, avg);
        } else {
            printf("%-10s %-10s %-10s %-10s\n", "-", "-", "-", "미입력");
        }
    }
}

// 조건 9. 4번 메뉴: 과목별 최고 득점자 확인
void print_top_scorers(student* list, int count) {
    if (count == 0) {
        printf("\n등록된 학생 정보가 없습니다.\n");
        return;
    }

    int max_py = -1, max_ja = -1, max_c = -1;
    int idx_py = -1, idx_ja = -1, idx_c = -1;

    for (int i = 0; i < count; i++) {
        if (list[i].grades.has_grades == 1) {
            if (list[i].grades.python > max_py) { max_py = list[i].grades.python; idx_py = i; }
            if (list[i].grades.java > max_ja) { max_ja = list[i].grades.java; idx_ja = i; }
        }
    }
}

```

```

    if (list[i].grades.c_lang > max_c) { max_c = list[i].grades.c_lang; idx_c
= i; }
}
}

```

```

printf("\n[4. 과목별 최고 득점자 확인]\n");
if (idx_py != -1) {
    printf("- 파이썬 최고점: %s (학번: %s) / %d점\n", list[idx_py].name,
list[idx_py].student_id, max_py);
} else {
    printf("- 파이썬: 성적이 입력된 학생이 없습니다.\n");
}

```

```

if (idx_ja != -1) {
    printf("- 자바 최고점: %s (학번: %s) / %d점\n", list[idx_ja].name,
list[idx_ja].student_id, max_ja);
}

```

```

if (idx_c != -1) {
    printf("- C언어 최고점: %s (학번: %s) / %d점\n", list[idx_c].name,
list[idx_c].student_id, max_c);
}
}

```

// 조건 10. 5번 메뉴: 정보 저장 (CSV 형태의 텍스트 파일로 저장)

```

void save_to_file(student* list, int count) {
    if (count == 0) {
        printf("\n저장할 학생 정보가 없습니다.\n");
        return;
    }
}

```

```

FILE *fp = fopen("student_grades.txt", "w");
if (fp == NULL) {
    printf("\n파일을 생성하거나 열 수 없습니다.\n");
    return;
}

```

```

fprintf(fp, "학번,이름,나이,파이썬,자바,C언어,평균\n");
for (int i = 0; i < count; i++) {
    fprintf(fp, "%s,%s,%d,", list[i].student_id, list[i].name, list[i].age);
    if (list[i].grades.has_grades == 1) {
        double avg = (list[i].grades.python + list[i].grades.java +
list[i].grades.c_lang) / 3.0;
        fprintf(fp, "%d,%d,%d,%.2f\n", list[i].grades.python,

```

```

list[i].grades.java, list[i].grades.c_lang, avg);
    } else {
        fprintf(fp, "-,-,-미입력\n");
    }
}

fclose(fp);
printf("\n-> 학생 정보와 성적이 'student_grades.txt' 파일로 성공적으로 저장되었습니다.\n");
}

int main() {
    // 조건 5. 구조체를 활용하여 동적으로 메모리 공간 할당 (최대 20명)
    student* student_list = (student *)malloc(sizeof(student) *
MAX_STUDENTS);
    if (student_list == NULL) {
        printf("메모리 할당에 실패했습니다.\n");
        return -1;
    }

    int student_count = 0;
    int choice;

    // 조건 11. 기능 수행 완료 후 다시 메인 메뉴 제공, 0 입력 시 종료
    while (1) {
        print_menu();
        scanf("%d", &choice);
        while (getchar() != '\n'); // 입력 버퍼 비우기

        switch (choice) {
            case 1:
                input_student_info(student_list, &student_count);
                break;
            case 2:
                input_grades(student_list, student_count);
                break;
            case 3:
                print_all_students(student_list, student_count);
                break;
            case 4:
                print_top_scorers(student_list, student_count);
                break;
            case 5:
                save_to_file(student_list, student_count);

```

```

break;
case 0:
printf("\n프로그램을 종료합니다.\n");
free(student_list);
return 0;
default:
printf("\n잘못된 입력입니다. 0~5 사이의 메뉴를 선택해주세요.\n");
}
}
return 0;
}

```

4. 소스코드 설명

[#define MAX_STUDENTS 20 및 구조체 정의]

매크로 상수를 정의하여 최대 입력 가능한 학생 수를 20명으로 제한하고 프로그램의 유지보수성을 높였다. 만약 추후에 관리할 학생 수를 더 늘리고 싶다면 20이 아닌 다른 숫자로 대체하면 된다. 또한 grad 구조체로 세 과목의 점수와 성적 입력 상태를 확인하는 플래그 has_grades 를 묶고, 이를 student 구조체 내부에 포함시켜 구조체 중첩을 구현하였다

[main() 함수]

malloc() 함수를 사용하여 MAX_STUDENTS 크기만큼 student 구조체 배열의 메모리를 동적으로 할당한다. while(1) 무한루프와 switch-case 문을 통해 사용자가 메뉴를 선택하고 각 기능의 함수를 호출할 수 있도록 프로그램의 흐름을 제어하며, 프로그램종료(0)를 선택하면 free() 함수를 호출하여 동적할당된 메모리를 안전하게 해제하여준다.

[input_student_info() 함수]

포인터로 전달받은 count 변수를 참조하여 현재 등록된 학생이 최대 인원(20명)을 초과하지 않는지 검사한다. 조건에 부합하면 학번, 이름, 나이를 차례로 입력받아 배열에 저장하고, 해당 학생의 성적이 아직 입력되지 않았음을 명시하기 위해 하위 구조체의 has_grades 멤버 변수를 0으로 초기화한 뒤 등록된 학생 수(count)를 1 증가시킨다.

[input_grades() 함수]

성적을 입력할 대상 학생의 학번을 문자열로 입력받고, for문과 문자열 비교 함수인 strcmp()를 이용해 배열을 순회하며 입력한 학번과 일치하는 구조체 인덱스를 탐색한다. 일치하는 학생을 찾으면 파이썬, 자바, C언어 점수를 각각 입력받아 멤버

변수에 저장하고, has_grades 플래그를 1로 변경하여 성적 입력이 완료되었음을 상태로 기록한다.

[print_all_students() 함수]

출력부에서는 %-15s, %-10d 등의 서식 지정자를 사용하여 좌측 정렬을 통해 각 열의 줄을 깔끔하게 맞춘다. has_grades가 1인(성적이 입력된) 학생을 탐색하면 3과목의 점수를 더한 뒤 3이 아닌 3.0으로 나누어 산술 연산의 결과를 실수형으로 얻기 위한 암묵적 형변환을 유도하고, 그 결과를 통해 평균을 출력한다. 성적이 없는 학생은 '-' 및 '미입력' 문구로 처리하여 예외 상황에 대비한다.

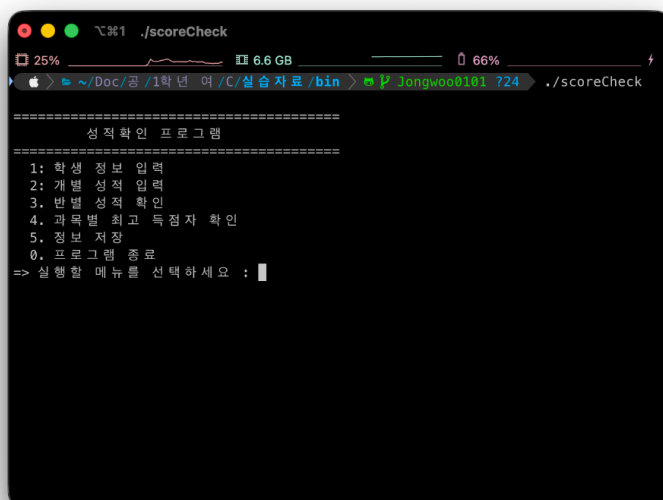
[print_top_scorers() 함수]

각 과목의 최고점을 저장할 변수(max_py 등)와 해당 학생의 인덱스를 저장할 변수(idx_py 등)를 -1로 초기화한다. for문을 이용해 0번 인덱스부터 순회하며 성적이 입력된 학생에 한해, 현재 저장된 최고점보다 더 높은 점수를 만나면 해당 점수와 인덱스로 값을 갱신한다. 루프가 모두 끝나면 최종적으로 찾아낸 인덱스를 이용해 과목별 최고 득점 학생의 이름, 학번, 점수를 출력한다.

[save_to_file() 함수]

fopen() 함수를 사용해 쓰기모드로 텍스트 파일을 생성하거나 연다. 반복문을 돌며 fprintf() 함수를 통해 배열에 저장된 학생들의 기본 정보와 성적, 평균 데이터를 쉼표 (,)로 구분하는 csv형식으로 파일에 차례대로 기록한다. 모든 정보 저장이 끝나면 메모리 누수를 방지하기 위해 fclose()로 파일을 닫아준다.

5. 실행결과



```
=====
성적 확인 프로그램
=====
1: 학생 정보 입력
2: 개별 성적 입력
3: 반별 성적 확인
4: 과목별 최고 득점자 확인
5: 정보 저장
0: 프로그램 종료
=> 실행할 메뉴를 선택하세요 : 1
```



```
. /scoreCheck

=====
1: 학생 정보 입력
2: 개별 성적 입력
3: 반별 성적 확인
4: 과목별 최고 득점자 확인
5: 정보 저장
0. 프로그램 종료
=> 실행할 메뉴를 선택하세요 : 1

[1. 학생 정보 입력]
학번을 입력하세요 : 2026591016
이름을 입력하세요 : JongwooWon
나이를 입력하세요 : 21
-> 학생 정보가 정상적으로 등록되었습니다.

=====
          성적 확인 프로그램
=====
1: 학생 정보 입력
2: 개별 성적 입력
3: 반별 성적 확인
4: 과목별 최고 득점자 확인
5: 정보 저장
0. 프로그램 종료
=> 실행할 메뉴를 선택하세요 : █
```

```
. /scoreCheck

=====
2: 개별 성적 입력
3: 반별 성적 확인
4: 과목별 최고 득점자 확인
5: 정보 저장
0. 프로그램 종료
=> 실행할 메뉴를 선택하세요 : 2

[2. 개별 성적 입력]
성적을 입력할 학생의 학번을 입력하세요 : 2026591016
-> JongwooWon 학생의 성적을 입력합니다.
파이썬 점수 : 100
자바 점수 : 100
C언어 점수 : 100
-> 성적 입력이 완료되었습니다.

=====
          성적 확인 프로그램
=====
1: 학생 정보 입력
2: 개별 성적 입력
3: 반별 성적 확인
4: 과목별 최고 득점자 확인
5: 정보 저장
0. 프로그램 종료
=> 실행할 메뉴를 선택하세요 : █
```

```

17% ██████████ 6.6 GB ██████████ 67% ██████████
성적 확인 프로그램
=====
1: 학생 정보 입력
2: 개별 성적 입력
3: 반별 성적 확인
4: 과목별 최고 득점자 확인
5: 정보 저장
0: 프로그램 종료
=> 실행할 메뉴를 선택하세요 : 3

[3. 반별 성적 확인]
학번      이름      나이  파이썬  자바      C언어      평균
-----
2026591016  JongwooWon  21    100     100     100     100.00

=====
성적 확인 프로그램
=====
1: 학생 정보 입력
2: 개별 성적 입력
3: 반별 성적 확인
4: 과목별 최고 득점자 확인
5: 정보 저장
0: 프로그램 종료
=> 실행할 메뉴를 선택하세요 : 
```

```

12% ██████████ 6.6 GB ██████████ 67% ██████████
성적 확인 프로그램
=====
1: 학생 정보 입력
2: 개별 성적 입력
3: 반별 성적 확인
4: 과목별 최고 득점자 확인
5: 정보 저장
0: 프로그램 종료
=> 실행할 메뉴를 선택하세요 : 4

[4. 과목별 최고 득점자 확인]
- 파이썬 최고점 : JongwooWon (학번 : 2026591016) / 100점
- 자바 최고점 : JongwooWon (학번 : 2026591016) / 100점
- C언어 최고점 : JongwooWon (학번 : 2026591016) / 100점

=====
성적 확인 프로그램
=====
1: 학생 정보 입력
2: 개별 성적 입력
3: 반별 성적 확인
4: 과목별 최고 득점자 확인
5: 정보 저장
0: 프로그램 종료
=> 실행할 메뉴를 선택하세요 : 
```

```
. /scoreCheck
11% 6.6 GB 67%
- C언어 최고점 : JongwooWon (학번 : 2026591016) / 100점

=====
성적 확인 프로그램
=====
1: 학생 정보 입력
2: 개별 성적 입력
3: 반별 성적 확인
4: 과목별 최고 득점자 확인
5: 정보 저장
0. 프로그램 종료
=> 실행할 메뉴를 선택하세요 : 5

-> 학생 정보와 성적이 'student_grades.txt' 파일로 성공적으로 저장되었습니다.

=====
성적 확인 프로그램
=====
1: 학생 정보 입력
2: 개별 성적 입력
3: 반별 성적 확인
4: 과목별 최고 득점자 확인
5: 정보 저장
0. 프로그램 종료
=> 실행할 메뉴를 선택하세요 : █
```

```
. /scoreCheck
6% 6.7 GB 68%

=====
성적 확인 프로그램
=====
1: 학생 정보 입력
2: 개별 성적 입력
3: 반별 성적 확인
4: 과목별 최고 득점자 확인
5: 정보 저장
0. 프로그램 종료
=> 실행할 메뉴를 선택하세요 : 2

[2. 개별 성적 입력]
성적을 입력할 학생의 학번을 입력하세요 : 100
-> 해당 학번의 학생을 찾을 수 없습니다.

=====
성적 확인 프로그램
=====
1: 학생 정보 입력
2: 개별 성적 입력
3: 반별 성적 확인
4: 과목별 최고 득점자 확인
5: 정보 저장
0. 프로그램 종료
=> 실행할 메뉴를 선택하세요 : █
```

```

=====
성적 확인 프로그램
=====
1: 학생 정보 입력
2: 개별 성적 입력
3: 반별 성적 확인
4: 과목별 최고 득점자 확인
5: 정보 저장
0: 프로그램 종료
=> 실행할 메뉴를 선택하세요 : 1000

잘못된 입력입니다. 0~5 사이의 메뉴를 선택해주세요.

=====
성적 확인 프로그램
=====
1: 학생 정보 입력
2: 개별 성적 입력
3: 반별 성적 확인
4: 과목별 최고 득점자 확인
5: 정보 저장
0: 프로그램 종료
=> 실행할 메뉴를 선택하세요 : 

```

```

=====
1: 학생 정보 입력
2: 개별 성적 입력
3: 반별 성적 확인
4: 과목별 최고 득점자 확인
5: 정보 저장
0: 프로그램 종료
=> 실행할 메뉴를 선택하세요 : 1000

잘못된 입력입니다. 0~5 사이의 메뉴를 선택해주세요.

=====
성적 확인 프로그램
=====
1: 학생 정보 입력
2: 개별 성적 입력
3: 반별 성적 확인
4: 과목별 최고 득점자 확인
5: 정보 저장
0: 프로그램 종료
=> 실행할 메뉴를 선택하세요 : 0

프로그램을 종료합니다.

```

```

scoreCheck.c  student_grades.txt ✕

bin > student_grades.txt
1   학번,이름,나이,파이썬,자바,C언어,평균
2   2026591016,JongwooWon,21,100,100,100,100.00
3

```